DocChat RAG spec

Scope of this spec: phases P0-P4 (backend only). Frontend, environments, hardening tests, complex documents and full evals are in ‘docs/backlog.md‘ under Later.

Goal
Turn the prototype in ‘src/backend/app‘ into a production-shaped agentic document-RAG backend. Users upload one or many documents, then summarize them, ask questions with citations, compare them, and query tables, through an agent that plans, calls read-only tools, grades its own retrieval, retries, and checks that the answer is grounded.

Non-goals
- Frontend, deployment environments, load/security test suites, OCR/scanned PDFs, complex table extraction, full-system evals (all Later).
- Multi-tenant billing, SSO, fine-tuning, writing or modifying user documents, any tool with side effects.

Users and flows
Single developer/user now; the API is built per-user (a ‘X-User-Id‘ header stands in for real auth until the Later phase).
1. Upload files (‘POST /documents‘) and receive ‘doc_id‘s. 2. List or delete them. 3. ‘POST /summarize‘ for a whole-document summary. 4. ‘POST /query‘ asks a question (P1: single-shot RAG; P3: agent, streamed over SSE). 5. Follow-ups reuse a ‘session_id‘ (P4).

Features (v1)
- P0: bug fixes, models loaded once, non-blocking, 5 endpoints, per-document ids and metadata, ‘llm‘ wrapper with mock mode, pydantic-settings config, uv with pinned deps, ‘prompts/*.md‘.
- P1: hybrid BM25 + dense retrieval, cross-encoder rerank, parent-child chunks with page/section metadata, citations (source id + span), "I don’t know" on weak retrieval, optional LLM compression, map-reduce summarization.
- P2: tools ‘search_documents‘, ‘summarize_document‘, ‘get_page_or_section‘, ‘list_documents‘, ‘query_table‘, ‘compare_documents‘; allow-listed, read-only, scoped to the caller’s ‘doc_id‘s.
- P3: LangGraph agent loop (router, plan, tool, grade, rewrite/retry, answer with citations, groundedness check); bounded by steps, tokens and USD; SSE step streaming; configurable provider.
- P4: per-session memory, prompt-injection defences, per-user rate limit and daily spend cap, PII-safe logging, ~20-item golden-set eval script.

Stack
Python 3.12, uv (pinned lockfile), FastAPI + Pydantic v2, pydantic-settings, Ruff, pytest + httpx. Chroma (embedded persistent client, no separate server), ‘sentence-transformers‘ (MiniLM embeddings, ‘bge-reranker-base‘), ‘rank-bm25‘, pandas, LangGraph. Hosted Claude via the ‘anthropic‘ SDK for the agent; Ollama optional. Exact dependency list is decided in the ADR; none is added without it.

Architecture
Layers: ‘api/‘ (routers, SSE) -> ‘services/‘ (ingestion, retrieval, summarization, agent) -> ‘core/‘ (config, llm wrapper, model registry, logging) -> ‘store/‘ (Chroma, doc registry, BM25 index). Models are loaded once in the FastAPI lifespan and held in a registry; CPU/blocking work runs in a threadpool. Agent tools call services, never the store directly. One doc per component plus an overall doc in ‘docs/architecture/‘, written in the PR that introduces the component.

Data model
- ‘Document‘: ‘doc_id‘, ‘owner_id‘, ‘filename‘ (original), ‘mime‘, ‘size‘, ‘sha256‘, ‘pages_or_sheets‘, ‘created_at‘, ‘status‘.
- ‘Chunk‘ (child, embedded) -> ‘parent_id‘ (larger context span): ‘chunk_id‘, ‘doc_id‘, ‘page|sheet‘, ‘section‘, ‘char_start‘, ‘char_end‘, ‘text‘.
- ‘Citation‘: ‘doc_id‘, ‘filename‘, ‘chunk_id‘, ‘page|sheet‘, ‘span‘.
- ‘Session‘ (P4): ‘session_id‘, ‘owner_id‘, bounded message history.

Registry is SQLite via the stdlib (no new server); Chroma holds vectors keyed by ‘doc_id‘ metadata. Re-uploading identical content (‘sha256‘) returns the existing ‘doc_id‘.

AI design (models, prompts, retrieval/agent loop, guardrails, cost per request)
Recorded line by line in ‘docs/ai-design.md‘ (rationale per row). Summary: embeddings ‘all-MiniLM-L6-v2‘ (local, free); rerank ‘bge-reranker-base‘ (local); generation default ‘claude-sonnet-5-5‘ for the agent and ‘claude-haiku-5-5‘ for map/grade/compress steps, model ids in config only; Ollama as the optional local provider. Prompts in ‘prompts/*.md‘, versioned. Retrieved text and tool output are data, never instructions. Answers must cite or say "I don’t know" when the top rerank score is under a threshold. Target cost: at most $0.05 per agent request, $0.02 per summarize call on a typical 20-page document.

Evals and targets (metric, dataset, threshold)
Golden set ‘evals/golden.jsonl‘ (~20 items, P4) over small fixture documents in ‘evals/fixtures/‘. Thresholds live in ‘evals/thresholds.json‘ (already set): recall@5 >= 0.8, MRR >= 0.6, faithfulness >= 0.85, relevance >= 0.8, citation_correct >= 0.9, agent task_success >= 0.8 with avg steps <= 8, injection block rate >= 0.9. Unit/integration tests use the mock LLM and run free. The full-architecture eval phase is Later.

Security and abuse
- Upload: size cap 10 MB (streamed, rejected before full read), extension allowlist, content sniff that must agree with the extension, filename sanitized, temp files always removed, no code execution from documents.
- Agent: allow-listed read-only tools, every tool call scoped to the caller's ‘doc_id‘s, bounded steps/tokens/USD per run, tool results treated as untrusted and wrapped in delimiters.
- Per-user rate limit and daily spend cap on every LLM endpoint. Logs never contain document text or prompts with PII; only ids, sizes, timings, tokens and USD.
- Secrets only from env, documented in ‘.env.example‘. Real key rotation for the committed ‘.env‘ is a human task (see Risks).

Performance targets
p95 latency <= 8 s for ‘/query‘ on a 20-page document with the hosted model; ingestion of a 20-page PDF <= 10 s on a laptop CPU; cold start loads models once (target <= 30 s); event loop never blocked by parsing or inference. Measured properly in the Later perf phase.

Deploy target and env vars

Local first: ‘uv run uvicorn‘, embedded Chroma at ‘CHROMA_DIR‘, SQLite at ‘DB_PATH‘. Containers and dev/prod environments are Later. Env vars (all in ‘.env.example‘): ‘ANTHROPIC_API_KEY‘, ‘LLM_PROVIDER‘ (‘anthropic|ollama|mock‘), ‘AGENT_MODEL‘, ‘FAST_MODEL‘, ‘OLLAMA_BASE_URL‘, ‘CHROMA_DIR‘, ‘DB_PATH‘, ‘MAX_UPLOAD_MB‘, ‘RATE_LIMIT_PER_MIN‘, ‘DAILY_USD_CAP‘, ‘AGENT_MAX_STEPS‘, ‘AGENT_MAX_USD‘.

Risks
- Committed ‘.env‘ contains an ‘OPEN_AI_API_KEY‘ and was read by tools earlier; treat as exposed and rotate (human).
- Mistral-7B-class local models are unreliable at tool calling, so Ollama is best-effort for the agent.
- Local cross-encoder plus embeddings add memory (~2 GB) and cold-start time; mitigated by loading once.
- Hosted-model spend: bounded by per-run and per-day caps; tests never call paid APIs.

Assumed defaults (not asked)
Hosted Claude for the agent, Haiku-class for sub-steps; embedded Chroma replaces the HTTP Chroma server; SQLite registry; header-based user id until real auth; 10 MB upload cap kept; no OCR; Python 3.12 and uv per the stack overlay; single-process deployment.